Report - Version 10: Decoder-Only Transformer

Introduction

Version 10 turns the explicit single Transformer block from Version 9 into a small decoder-only Transformer language model built from reusable components.

The same 440-character English corpus, 27-character vocabulary, four-character context, 8-dimensional character and positional embeddings, causal next-character objective, reproducible train/validation split, mini-batch optimization and autoregressive generation are preserved. The main change is architectural and organizational: one reusable TransformerBlock is instantiated twice, the blocks use independent parameters, and the output of the first block becomes the input of the second.

```
Characters
  |
Integer identifiers
  |
4-character context + 4 sequential targets
  |
Character + positional embeddings
  |
Transformer block 1
  |
Transformer block 2
  |
Vocabulary logits at every position
  |
Generated text
```

Goal of Version 10

- preserve the same corpus, vocabulary, four-character context and training objective from Version 9;
- group the attention, residual, LayerNorm and feed-forward operations into a reusable Transformer block;
- stack two independent Transformer blocks while preserving causal information flow;
- group model parameters so training no longer passes every Transformer variable manually;
- keep the mathematics explicit while making the implementation easier to extend;
- verify block independence, shapes, parameter count, causal masking, gradients, updates, reproducibility and generation.

Architecture

The complete Version 10 flow is:

```
Corpus: 440 characters
  |
Vocabulary: 27 characters
  |
436 sequential context-target examples
  |
Input tensor:  (436, 4)
Target tensor: (436, 4)
  |
Character embeddings: (27, 8) + positions: (4, 8)
  |
Transformer block 1: (4, 8) -> (4, 8)
  |
Transformer block 2: (4, 8) -> (4, 8)
  |
Vocabulary logits: (4, 27) per example
  |
Reproducible 80 / 20 split + mini-batch updates
  |
Sliding-window autoregressive generator
```

Reusable Transformer blocks

Each Transformer block receives a sequence representation with shape (batch, 4, 8). Inside the block, queries, keys and values are projected to 16 dimensions, causal scaled dot-product attention mixes only the current and earlier positions, and an output projection returns the attention result to 8 dimensions. Residual connections, LayerNorm and a position-wise 8 -> 16 -> 8 feed-forward network preserve the representation shape.

The two blocks have the same structure but independent trainable variables. The second block therefore processes the representation already transformed by the first block rather than repeating the same parameters.

Context windows, split and mini-batches

```
training examples   = 348
validation examples = 88
batch size          = 32
updates per epoch   = 11
```

Only training examples are shuffled and updated. Validation measures loss without changing the parameters. Neighboring windows overlap, so this split remains a lightweight learning monitor rather than a strong estimate of generalization to unseen text.

Trainable parameters

The model contains 23 trainable TensorFlow variables and exactly 2064 trainable parameters:

```
Character embeddings             27 x 8   = 216
Positional embeddings              4 x 8   =  32
Transformer block 1                          = 800
Transformer block 2                          = 800
Output weights                     8 x 27  = 216
-----------------------------------------------
Total                                        = 2064
```

Each Transformer block contributes exactly 800 parameters:

```
Query projection                 8 x 16 = 128
Key projection                   8 x 16 = 128
Value projection                 8 x 16 = 128
Attention output projection     16 x 8  = 128
LayerNorm 1 scale + shift        8 + 8   =  16
Feed-forward input               8 x 16 = 128
Feed-forward output             16 x 8  = 128
LayerNorm 2 scale + shift        8 + 8   =  16
-----------------------------------------------
Per block                                  = 800
```

Transformer block details

For input X, each block computes its own query, key and value projections. The causal mask is applied before softmax, and the resulting contextual values are projected back to the 8-dimensional model representation:

```
Q = X Wq
K = X Wk
V = X Wv

scores = Q K^T / sqrt(dk)
masked_scores = causal_mask(scores)
A = softmax(masked_scores)
H = A V
```

No bias terms are used in the attention, feed-forward or vocabulary projections. Both LayerNorm operations use trainable scale and shift parameters.

Causal mask

```
[[1, 0, 0, 0],
 [1, 1, 0, 0],
 [1, 1, 1, 0],
 [1, 1, 1, 1]]
```

Loss, training and reproducibility

Sparse softmax cross-entropy is computed at all four sequence positions. The per-position losses have shape (batch, 4) and their mean is the scalar objective. tf.GradientTape calculates gradients for all 23 trainable variables. Embedding lookups can produce sparse IndexedSlices gradients; because this model is very small, they are converted to dense tensors before the manual mini-batch gradient-descent update.

```
Learning rate: 1.0
Batch size:    32
Epochs:        100
Seed:          42
```

Saved training result

```
Initial train loss:       3.295300
Final train loss:         1.844497
Initial validation loss:  3.291214
Final validation loss:    2.177593
Best validation loss:     2.126416
Best validation epoch:    98
```

Training loss decreases substantially. Validation also improves and reaches its best value near the end of training. The deeper model obtains slightly lower saved losses than Version 9 in this run, but the tiny overlapping dataset and single seed do not support a strong generalization comparison. Small numerical differences can appear across TensorFlow environments.

Inspection of learned probabilities

For context mode, the highest final-position probabilities in the saved run are:

```
space  0.3901
t      0.1992
l      0.1369
r      0.0717
c      0.0697
s      0.0373
```

Inspection of stacked causal attention

```
Block 1
       m       o       d       e
m  1.0000  0.0000  0.0000  0.0000
o  0.4487  0.5513  0.0000  0.0000
d  0.3645  0.2860  0.3495  0.0000
e  0.3506  0.1381  0.3166  0.1948

Block 2
       m       o       d       e
m  1.0000  0.0000  0.0000  0.0000
o  1.0000  0.0000  0.0000  0.0000
d  0.0410  0.0748  0.8842  0.0000
e  0.9923  0.0000  0.0077  0.0000
```

Every row sums to approximately 1 and the maximum future attention is exactly 0.0 in both blocks, confirming that stacking preserves the causal constraint.

Autoregressive generation

Generation uses a sliding four-character window. At every step the complete context passes through both Transformer blocks. Only the logits from the final sequence position are converted to probabilities and sampled, then the oldest context character is discarded and the new character is appended.

With seed 42, five independent samples from context mode in the saved execution are:

```
s, c, t, r, space
```

The 300-character demonstration begins:

```
modes spaurs matsin.
atindprs macorte macod nspesinct.
asiond caned ngom inet.
eromel s adve math inan tunctong comake spredin m ne coms le tin...
```

Included checks

- the input and target tensors both have shape (436, 4);
- the model contains exactly two Transformer blocks with independent trainable variables;
- all 23 trainable variables have the expected shapes and total exactly 2064 parameters;
- both blocks preserve the (batch, 4, 8) representation and produce 4 x 4 attention matrices;
- future attention weights are exactly zero in both blocks and every attention row sums to 1;
- next-character probability distributions are finite and sum to 1;
- all gradients are present and finite, and every trainable variable changes during training;
- training and validation indices are disjoint and recorded loss histories remain finite;
- retraining and autoregressive generation are reproducible with the same seed.

The notebook finishes its test section with All checks passed.

What Version 10 teaches

Version 10 is the first stage in the project where the Transformer architecture is organized as a reusable decoder-only stack rather than a single explicit block. The key conceptual progression is:

```
context -> embeddings + positions
        -> Transformer block 1
        -> Transformer block 2
        -> vocabulary logits at every position
        -> loss over all positions
        -> final-position autoregressive sampling
```

The important distinction from Version 9 is not a new attention equation. The mathematics inside each block is preserved, while model depth and software structure change: parameters are grouped into reusable components and independent blocks can be stacked sequentially.

Limitations

- the context length remains fixed at four characters and the corpus contains only 440 characters;
- the model remains character-level and is intentionally far smaller than a large language model;
- each Transformer block uses one attention mechanism rather than multi-head attention;
- only two Transformer blocks are used; there is no dropout or learning-rate schedule;
- overlapping windows appear across the random train/validation split, so validation is not a robust generalization benchmark;
- generation retains the final epoch parameters rather than automatically restoring the best validation checkpoint;
- the implementation remains deliberately explicit and does not yet use a higher-level Keras model abstraction.

Conclusion

Version 10 successfully turns the single causal Transformer block from Version 9 into a small decoder-only Transformer composed of two reusable, independent blocks. It preserves the inspectable character-level setting while introducing the model organization and depth needed for the next stage of the project.

The model is still intentionally tiny, but the core decoder-only structure is now in place. A later version can move from character tokens toward subword tokenization and larger documented datasets while preserving the architecture developed here.